Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Штыков М.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 13.10.2026

Москва, 2026

Постановка задачи

Вариант 3. Отсортировать массив целых чисел при помощи параллельной сортировки слиянием. Реализовать последовательную и многопоточную версии на C, ограничить число потоков ключом запуска, продемонстрировать их количество средствами ОС и исследовать время, ускорение и эффективность.

Общий метод и алгоритм решения

Массив рекурсивно делится пополам до участков из одного элемента. Отсортированные половины объединяются во временном массиве, после чего результат копируется обратно. Последовательная версия выполняет обе ветви по очереди.

В параллельной версии функция work получает границы участка и бюджет потоков. Левая половина обрабатывается новым потоком, правая — текущим. После pthread_join выполняется слияние. При бюджете 1 участок сортируется последовательно. Это рекурсивная модель fork–join.

Ключ -t P задаёт максимальное общее число потоков, включая основной. Бюджет делится между двумя ветвями, поэтому лимит не превышается. Для P > N он уменьшается до N; для пустого массива остаётся один поток.

Использованные системные интерфейсы

pthread_create — создание потока; pthread_join — ожидание его завершения; clock_gettime(CLOCK_MONOTONIC) — измерение прошедшего времени. sysconf используется скриптом экспериментов для определения числа логических процессоров.

Особенности реализации

Потоки записывают данные в непересекающиеся участки основного и временного массивов. Родительское слияние начинается после завершения обеих ветвей, поэтому мьютекс не требуется. Последнее слияние выполняется одним потоком и ограничивает ускорение. Последовательная сложность — O(N log N), вспомогательный массив — O(N).

Код программы

Ключевой фрагмент parallel.c; полные исходники: sequential.c, parallel.c и common.h.

```
pthread_t thread;
thread_error(pthread_create(&thread, NULL, work, &left),
        "pthread_create");
work(&right);
thread_error(pthread_join(thread, NULL), "pthread_join");
merge(args->a, args->tmp, args->left, mid, args->right);
```

Протокол работы программы

Среда: Ubuntu в WSL2, Linux 6.6.87.2-microsoft-standard-WSL2, x86_64; 12 логических процессоров. Дата экспериментов: 13.10.2026. Сборка GCC, стандарт C99, оптимизация -O2.

```
gcc -std=c99 -O2 -Wall -Wextra -Wpedantic sequential.c -o sequential
gcc -std=c99 -O2 -Wall -Wextra -Wpedantic -pthread parallel.c -o parallel
```

Проверка сортировки

В обе программы введён массив из 8 элементов: 5 −2 9 0 5 1 −7 3. Получен результат: −7 −2 0 1 3 5 5 9. На массиве из 1 000 000 случайных чисел предварительный запуск дал около 144 мс для последовательной версии и 77 мс для четырёх потоков. Эти одиночные замеры не подставлялись в итоговые таблицы.

Подтверждение количества потоков

```
./parallel -n 1000000 -t 4 -q -d &
pid=$!
sleep 1
ps -T -p "$pid" -o pid,spid,stat,comm
ps -o pid,nlwp,comm -p "$pid"
wait "$pid"
```

В выводе ps для процесса PID 13497 отображены четыре SPID: 13497, 13503, 13504, 13505; NLWP = 4. Это подтверждает создание основного и трёх дополнительных потоков. Режим -d содержит задержку около 10 с и в измерениях производительности не используется.

```
[1] 13497
PID=13497; demonstration delay is included in time
    PID    SPID STAT COMMAND
  13497   13497 Sl   parallel
  13497   13503 Sl   parallel
  13497   13504 Sl   parallel
  13497   13505 Sl   parallel
    PID NLWP COMMAND
  13497    4 parallel
n=1000000 threads=4 time_ms=10090.570147 verified=yes demo=1
[1]+  Done                    ./parallel -n 1000000 -t 4 -q -d
```

Методика экспериментов и показатели

Команда python3 benchmark.py > results.md запускает две версии на одинаковых исходных данных. Для каждого режима выполнены прогрев и три измерения; в таблицах приведены медианы. Seed: 42, 43, 44. Измеряются сортировка, создание и ожидание потоков; подготовка массива и вывод исключены.

Проверены N = 100 000 и 1 000 000, четыре вида данных. P = 1, 2, 3, 4 меньше 12; P = 12 равно числу логических процессоров; P = 48, 96, 192 значительно больше него.

Ускорение S(P) = Tпосл / TP; эффективность E(P) = S(P) / P × 100%. Tпосл — время последовательной версии, TP — время параллельной, P — лимит потоков с учётом основного. Ускорение показывает выигрыш по времени, эффективность — отношение ускорения к выделенному числу потоков, а не загрузку CPU.

Результаты для 100 000 элементов

Случайные числа. Тпосл = 14,072 мс.

Потоки P	Время TP, мс	Ускорение S	Эффективность, %
1	11,499	1,224	122,38
2	8,305	1,694	84,71
3	8,477	1,660	55,33
4	4,880	2,883	72,08
12	6,102	2,306	19,22
48	8,319	1,691	3,52
96	11,771	1,195	1,25
192	20,432	0,689	0,36

Возрастающий порядок. Тпосл = 4,391 мс.

Потоки P	Время TP, мс	Ускорение S	Эффективность, %
1	2,367	1,855	185,49
2	2,193	2,002	100,10
3	1,640	2,678	89,28
4	1,860	2,361	59,01
12	2,165	2,028	16,90
48	5,469	0,803	1,67
96	9,854	0,446	0,46
192	22,293	0,197	0,10

Убывающий порядок. Тпосл = 2,902 мс.

Потоки P	Время TP, мс	Ускорение S	Эффективность, %
1	2,058	1,410	140,99
2	2,998	0,968	48,41
3	1,720	1,688	56,26
4	1,426	2,035	50,86
12	1,818	1,596	13,30
48	5,695	0,510	1,06
96	10,362	0,280	0,29
192	26,167	0,111	0,06

Одинаковые числа. Тпосл = 4,574 мс.

Потоки P	Время TP, мс	Ускорение S	Эффективность, %
1	4,096	1,117	111,67
2	2,158	2,119	105,97
3	2,485	1,841	61,37
4	1,745	2,621	65,52
12	3,123	1,465	12,21
48	7,284	0,628	1,31
96	12,877	0,355	0,37
192	34,097	0,134	0,07

Для случайного массива лучшее измеренное время — 4,880 мс при P = 4 (S = 2,883). При P = 192 время выросло до 20,432 мс, превысив последовательное. Для трёх остальных типов входа при 48–192 потоках также наблюдается замедление относительно последовательной версии.

Результаты для 1 000 000 элементов

Случайные числа. Tпосл = 164,115 мс.

Потоки P	Время TP, мс	Ускорение S	Эффективность, %
1	160,821	1,020	102,05
2	95,349	1,721	86,06
3	89,274	1,838	61,28
4	60,428	2,716	67,90
12	52,985	3,097	25,81
48	48,176	3,407	7,10
96	59,017	2,781	2,90
192	62,984	2,606	1,36

Возрастающий порядок. Tпосл = 30,125 мс.

Потоки P	Время TP, мс	Ускорение S	Эффективность, %
1	34,710	0,868	86,79
2	19,674	1,531	76,56
3	17,573	1,714	57,14
4	14,714	2,047	51,18
12	15,752	1,912	15,94
48	16,357	1,842	3,84
96	23,690	1,272	1,32
192	50,848	0,592	0,31

Убывающий порядок. Tпосл = 37,007 мс.

Потоки P	Время TP, мс	Ускорение S	Эффективность, %
1	30,627	1,208	120,83
2	19,018	1,946	97,29
3	20,143	1,837	61,24
4	16,624	2,226	55,65
12	16,461	2,248	18,74
48	19,633	1,885	3,93
96	20,668	1,791	1,87
192	33,051	1,120	0,58

Одинаковые числа. Tпосл = 33,408 мс.

Потоки P	Время TP, мс	Ускорение S	Эффективность, %
1	32,271	1,035	103,52
2	17,517	1,907	95,36
3	18,061	1,850	61,66
4	16,303	2,049	51,23
12	16,015	2,086	17,38
48	19,121	1,747	3,64
96	23,614	1,415	1,47
192	36,215	0,922	0,48

Вывод

Реализованы последовательная и параллельная сортировки слиянием на C с POSIX Threads. Количество потоков подтверждено средствами ОС. Для миллиона случайных чисел минимум среди проверенных режимов — 48,176 мс при P = 48: S = 3,407, E = 7,10%. При P = 12 эффективность выше (25,81%), хотя время больше (52,985 мс). Избыток потоков не даёт пропорционального ускорения.

Результат зависит от размера и порядка данных: для миллиона возрастающих чисел минимум получен при P = 4, для обратного порядка и одинаковых чисел — при P = 12. Выигрыш ограничивают последовательные слияния, затраты на потоки и конкуренция за ресурсы.

Значения E > 100% при P = 1 не отражают параллелизм: в этом режиме новые потоки не создаются. Разница до 1,855 раза указывает на существенную вариативность отдельных замеров. Возможные

причины — нагрузка WSL2 и основной ОС, частота CPU, кэш и различия компиляции. Три повтора не позволяют установить причину или считать близкие результаты устойчивыми.